

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

You are creating a balancing system for an online competitive game. Every player's account has a unique rank, although not every rank is represented (since only a fraction of players are online at any given time). Player ranks are stored in an array A of size n , consisting of unique and not necessarily consecutive positive integers. Older accounts are listed before newer ones.

To help balance the game, new players are given a numerical bonus based on the number of accounts that are both older than theirs, *and* have a higher rank. You need to find an efficient way to calculate the bonuses for every player. That is, you want to create an array $S = [S_1, S_2, \dots, S_n]$ where S_i is the number of players in A to the left of index i with a value greater than $A[i]$.

Formally, $S_i = \text{number of } j \text{ s.t. } 1 \leq j < i \text{ and } A[j] > A[i]$. Develop an efficient algorithm to create S from any A .

Example:

Input: $A = [10, 5, 12, 1, 11]$

Output: $S = [0, 1, 0, 3, 1]$

Presentation 1 Solution

Pseudocode

Algorithm 1 Balancing Score Array

```
procedure FindBalanceScores( $A, n$ )
     $S \leftarrow$  array of length  $n$  with all elements 0
    MODIFIED-MERGESORT( $A, 1, n, S$ )
    return  $S$ 
end procedure

procedure modified-mergesort( $A, start, end, inv$ )
    if  $start = end$  then
        return [ $start$ ]
    else
         $mid \leftarrow \lfloor \frac{start+end}{2} \rfloor$ 
        L-indices  $\leftarrow$  MODIFIED-MERGESORT( $A, start, mid, inv$ )
        R-indices  $\leftarrow$  MODIFIED-MERGESORT( $A, mid + 1, end, inv$ )
        return MERGE( $A, start, mid, end$ , L-indices, R-indices,  $inv$ )
    end if
end procedure

procedure merge( $A, start, mid, end$ , L-ind, R-ind,  $inv$ )
     $n_1, n_2 \leftarrow (mid - start + 1), (end - mid)$ 
     $B, C \leftarrow$  copy of ( $A[start..mid]$ ), copy of ( $A[mid + 1..end]$ )
     $ind \leftarrow$  array of length  $n_1 + n_2$ 
     $B[n_1 + 1], C[n_2 + 1] \leftarrow \infty$ 
     $i, j \leftarrow 1$ 
    for  $k \leftarrow start$  to  $end$  do
        if  $B[i] \leq C[j]$  then
             $A[k] \leftarrow B[i]$ 
             $ind[(i + j) - 1] \leftarrow$  L-ind[ $i$ ]
             $i \leftarrow i + 1$ 
        else
             $A[k] \leftarrow C[j]$ 
             $inv[\text{R-ind}[j]] \leftarrow inv[\text{R-ind}[j]] + (n_1 - i + 1)$ 
             $ind[(i + j) - 1] \leftarrow$  R-ind[ $j$ ]
             $j \leftarrow j + 1$ 
        end if
    end for
    return  $ind$ 
end procedure
```

Proof of Correctness: modified-mergesort

Inductive Statement: $A[start..end]$ is sorted and $ind[start..end]$ holds the original indices of the corresponding sorted elements before any sorting occurred.:

1. $A[start..end]$ is sorted.
2. For all $p \in (\text{L-indices} \cup \text{R-indices})$, $inv[p] = \text{number of pairs } (i, p) \text{ s.t. } A[i] > A[p] \text{ and } i < p$.
3. The value **returned** is array ind s.t. $ind[i] = \text{the original index of } A[i] \text{ before any sorting occurred}$.

Base Case: When $start = end$, the length of the subarray being considered is 1. Therefore, the subarray is always “sorted”, and ind correctly contains only the index of the element (which is $start$). The balancing score of the single element doesn’t change from its default value of 0 since there are no other elements to compare it against.

Inductive Step: Assume the inductive statement holds for some $end - start + 1 \geq 1$. This means the calls to **modified-mergesort** in the body of the **else** branch guarantee that $A[start..mid]$ and $A[mid + 1..end]$ are both sorted, since the subarrays are within $[end...start + 1]$. For the same reason, L-indices and R-indices will correctly contain the original indices of elements in the two subarrays before sorting occurred.

We prove below that the call to **MERGE** results in $A[start..end]$ being sorted, and that the values ind that the function **returns** are the correct original indices of elements in $A[start..end]$ before sorting occurred.

Recall, a pair (i, j) contributes to the score value $S[j]$ iff $A[i] > A[j]$ and $i < j$. All contributing pairs can be split into 3 distinct regions:

1. $1 \leq i < j < \frac{n}{2}$
2. $\frac{n}{2} < i < j \leq n$
3. $i \leq \frac{n}{2} < j \leq n$

According to the inductive assumption, the first two calls of **modified-mergesort** evaluate all the potential contributors in Inequalities 1 and 2. We will prove in the following section, that the call to **merge** evaluates all of the potential contributors from the third inequality. Therefore, the scores for the original elements are calculated correctly assuming that $A[start..end]$ are the only elements in question.

Therefore, **modified-mergesort** correctly calculates the score array S for any array A of arbitrary finite size.

Proof of Correctness: merge

Loop Invariant

1. $A[start..k-1]$ is sorted and contains the $k - start$ smallest elements from both B and C
 2. $A[start..k-1] \leq B[i..n_1+1], C[j..n_2+1]$
 3. $ind[1..(i+j)-2]$ contains the original indices of the elements in A before sorting
 4. For $p = ind[1], \dots, ind[(i+j)-2]$, $inv[p] = \text{number of pairs } (i, p) \text{ s.t. } A[i] > A[p] \text{ and } i < p$.
- **Initialization** Before the loop, $k = start$, meaning that the subarray $A[start..start-1]$ does not exist, making statement (1) and (2) vacuously true. Also, $i = 1, j = 1$, meaning that subarray $ind[1..0]$ does not exist, so statements (3) and (4) are vacuously true.
 - **Maintenance** Assume the L.I. holds for some arbitrary iteration of the loop. Important to note is that, based on the behavior of **modified-mergesort**, all elements in array L-ind will be less than all elements in array R-ind. There are two cases to consider:

Case 1: $B[i] \leq C[j]$

Since $L-ind[i] < R-ind[j]$ and $A[start + i - 1] < A[mid + j]$, this pair of numbers does not follow the definition of a candidate pair. Therefore, keeping $inv[L-ind[i]]$ the same value ensures that statement (4) holds. After assigning $A[k] = \min\{B[i], C[j]\}$ and incrementing i , statements (1) and (2) will hold. Finally, the code places the original index of element $B[i]$ into $ind[(i+j)-1]$, so statement (3) holds.

Case 2: $B[i] > C[j]$

Statements (1), (2), and (3) hold by a similar argument made in Case 1. However, since $B[i] > C[j]$ and $L-ind[i] < R-ind[j]$, this means we have encountered a pair of elements that can affect a score. Since $B[i] \leq B[i+1..n_1]$, that also means all pairs $(L-ind[l], R-ind[j])$ affect scores for $l = i+1, \dots, n_1$.

We need to be careful though: when $B[i] = \infty$ we **do not want to count this pair in any score** since $B[i]$ is just a sentinel element (i.e. it doesn't exist in A , it's just convenient to include for the merge procedure). This situation happens when $i = n_1 + 1$; when we plug that expression into the value we update $inv[R-ind[j]]$ by, we get

$$(n_1 - i + 1) \implies (n_1 - (n_1 + 1) + 1) = 0$$

which ensures that this situation doesn't actually change the score of element $C[j]$. Therefore we can ensure that statement (3) is correct.

- **Termination** When the loop terminates, $k = \text{end} + 1$. By Statement (1), $A[\text{start}..\text{end}]$ is sorted and contains the $(\text{end} - \text{start} + 1)$ smallest elements from both B and C (which together contain all of the elements in $A[\text{start}..\text{end}]$). This means we have achieved the goal of sorting subarray $A[\text{start}..\text{end}]$.

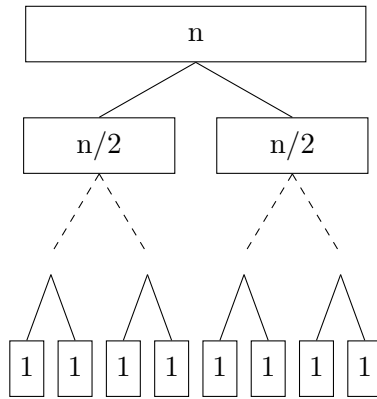
Since $B[n_1 + 1] = C[n_2 + 1] = \infty$, it was impossible for i to be $> n_1 + 1$ or j to be $> n_2 + 1$ at any point during the loop. Therefore, at termination, $i = n_1 + 1, j = n_2 + 1$ since either only i or only j increments at each iteration. After substituting these facts into statement (3), we get “ $\text{ind}[1..n_1 + n_2]$ contains the original indices of corresponding elements in A before A was sorted”, which achieves the goal of keeping track of elements’ original indices.

Finally, we see that for $p = \text{ind}[1], \dots, \text{ind}[n_1 + n_2]$, $\text{inv}[p] = \text{number of pairs } (i, p) \text{ s.t. } A[i] > A[p] \text{ and } i < p$. This means all score candidate pairs for subarray $A[\text{start}..\text{end}]$ that contain one element from B and one element from C have been accounted for. This fact is useful in the proof for **modified-mergesort**.

Runtime

Either Master’s Method or use of a recursion tree will suffice in analyzing the runtime. Since Master’s Method has not been taught yet at this point in the semester, this solution will use the recursion tree.

We know that each call to **modified-mergesort** will result in two recursive calls, each on an array of $n/2$ length. A tree diagram has been given below to encapsulate this behavior. At this point, we need to calculate the time complexity of the **merge** procedure in order to determine the work at each node of the recursion tree.



Level	Nodes/Level	Work/Node	Work/Level
0	1	$7n/2$	$7n/2$
1	2	$7n/4$	$7n/2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
k-1	$n/2$	7	$7n/2$
k	n	1	n

Table 1: Recursion Tree Table

Notice, if there are p nodes in some level of the tree, then there will be $p/2$ calls to **merge**, since the procedure takes in pairs of nodes (which are sorted subarrays in the context of the problem).

Before the loop in **merge**, elements from $A[start..end]$ are copied into two arrays B, C . If we define $end - start + 1 = n$, then this operation takes n time. During each iteration of the loop, we store a new element in A and ind , so this will take $2n$ time. Finally, every time we load an element from C into A , we update its score. Since all elements from C are added into A , this takes $n/2$ time. So, in total our **merge** method takes $7n/2$ time.

Normally, this could just be treated as $\mathbf{O}(n)$. However we want to directly insert this value into the **Work/Node** column of the recursion tree table. From there,

$$(\mathbf{Work/Node}) * (\mathbf{Nodes/Level}) = (\mathbf{Work/Level})$$

so the rightmost column of the recursion tree table can be filled out. All that needs to be done is multiply the **Work/Level** by the number of levels, k . We can calculate k by noticing the number of nodes at each decreasing level is divided by 2. Therefore,

$$\begin{aligned} 2^k &= n \\ k &= \log(n) \end{aligned}$$

So the total work done can be described by

$$\text{Work} = n + \frac{7n}{2}(\log(n) - 1) = \Theta(n \log n)$$

Presentation 2

You are once again working with a small independent comics publisher to digitize and organize their archives. This section of the archive was originally sorted in chronological order, with issues starting from #1 and continuing upward (although there may be missing issues). However, during transport to a new building, the ascending order has been shifted so that Issue #1 is somewhere in the middle of the sequence. For example, the issues may have started out in the order $[1, 2, 3, 5, 6, 7, 8]$, but is now in the order $[5, 6, 7, 8, 1, 2, 3]$.

Your task is to come up with a way to quickly find a specific issue number in this shifted archive. Given the list of issue numbers and the target issue number, return the index of the target issue. If the target issue is missing from the archive, return -1. Develop an efficient algorithm to solve this problem.

Example:

Input:

- Shifted Archive: $[5, 6, 7, 8, 1, 2, 3]$
- Target Issue: 6

Output: 2

Presentation 2 Solution

Pseudocode

Algorithm 2 Search in Rotated Sorted Array

```
function SEARCHINROTATEDARRAY(nums, target)
    left  $\leftarrow$  0
    right  $\leftarrow$  length(nums) - 1
    while left  $\leq$  right do
        mid  $\leftarrow$   $\lfloor (left + right) / 2 \rfloor$ 
        if nums[mid] = target then
            return mid
        end if
        if nums[left]  $\leq$  nums[mid] then
            if nums[left]  $\leq$  target < nums[mid] then
                right  $\leftarrow$  mid - 1
            else
                left  $\leftarrow$  mid + 1
            end if
        else
            if nums[mid] < target  $\leq$  nums[right] then
                left  $\leftarrow$  mid + 1
            else
                right  $\leftarrow$  mid - 1
            end if
        end if
    end while
    return -1
end function
```

Proof of Correctness

Loop Invariant:

If the target exists in the array, it must be within the current bounds defined by $A[left..right]$.

- **Initialization**

We have the initial values $left = 0$ and $right = \text{length}(\text{nums}) - 1$. At this point, the search space includes the entire array, so if the target exists in the array, it is guaranteed to be within this range. Therefore, the invariant holds before the first iteration.

- **Maintenance** Assume the loop invariant holds at some arbitrary iteration. During each iteration, we compute the midpoint: $mid = \lfloor (left + right) / 2 \rfloor$

There are three main cases for how we update $left$ and $right$:

Case 1: $nums[mid] = target$

We have found the target, so the function returns mid . Therefore, no more iterations of the loop will complete, and the goal of the algorithm has been achieved.

Case 2: $nums[left] \leq nums[mid]$

We know the original array before the rotation was sorted in ascending order. After rotation, it has at most one point where the order is broken, which we call a pivot. If the pivot were to exist in $nums[left..mid]$, then that would mean $nums[left] > nums[mid]$. However, in this case, $nums[left] \leq nums[mid]$, which means that $nums[left..mid]$ does not contain the pivot and is sorted in ascending order.

If the target's value is in between the bounds of the sorted array $nums[left..mid]$, i.e. $nums[left] \leq target < nums[mid]$, then we can guarantee that the target, if it exists, must be in the subarray $nums[left..mid - 1]$. Therefore, we update $right = mid - 1$.

Otherwise, the target is not in between the bounds of the sorted array $nums[left..mid]$, and if it exists must be in the right half. So, we update: $left = mid + 1$.

In both cases, we reduce the search space, and the invariant holds because the target must be within the updated range if it exists.

Case 3: $nums[left] > nums[mid]$

Use similar reasoning to Case 2. If $nums[mid] \leq nums[right]$, then no pivot exists in subarray $nums[mid..right]$. We then check if the target's value is in between the bounds of the right half, i.e. $nums[mid] < target \leq nums[right]$. If the target is in the right half, we update $left = mid + 1$. Otherwise, the target, if it exists, must be in the left half, so we update: $right = mid - 1$.

Again, we reduce the search space, and the invariant holds because the target must be within the updated range if it exists.

Thus, after each iteration of the loop, the search space is halved, and the invariant is maintained: if the target is in the array, it is contained within the range $A[left..right]$.

- **Termination** The loop terminates when $nums[mid] = target$ at some iteration or when $left > right$. In the first case, our algorithm successfully achieves the goal of the program and terminates. Otherwise, when $left > right$, the search space has been reduced to nothing. Therefore, The target does not exist in the array, and the function correctly returns -1 .

Runtime Analysis

We know that the array has n elements. After the first comparison, we eliminate half of the array, so we are left with $\frac{n}{2}$ elements. After the second comparison, we eliminate half of the remaining half of the array, so we are left with $\frac{n}{4}$ elements. We continue this on to finally the k -th iteration, when we have reduced our search space to a size of 1.

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log(n)$$

k represents the maximum number of iterations made by the algorithm. Each iteration of the while loop body takes constant time, as it is just comparisons and variable updates. Thus, the worst-case runtime is: $\mathbf{O}(\log(n))$